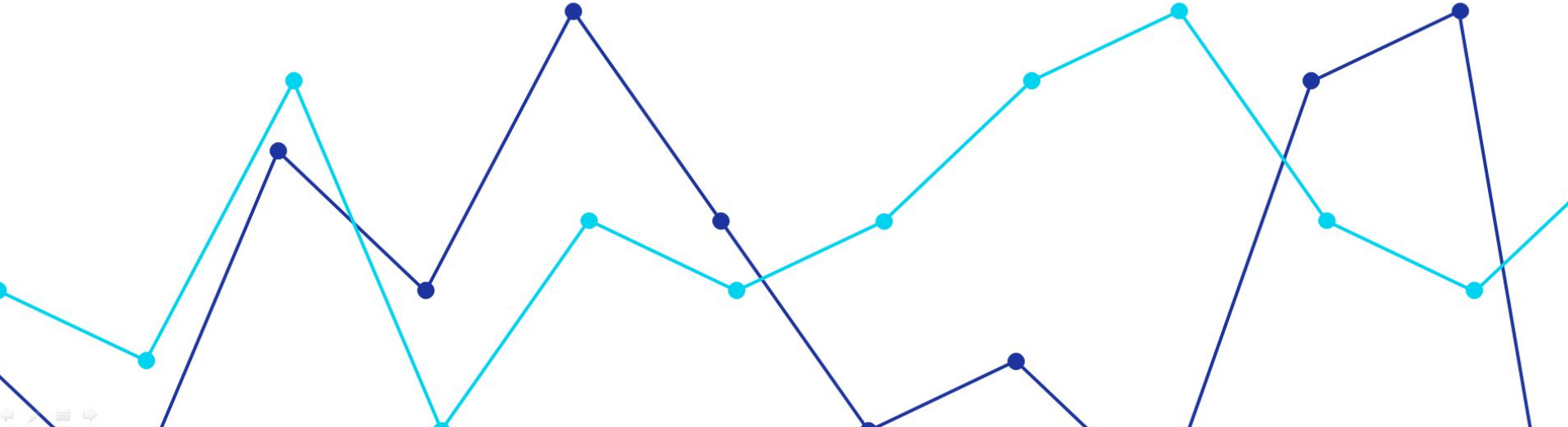
DTSA 5509 - Final Project

Shares of Online News Prediction



Data Source

Online News Popularity Data Set

UCI

Machine Learning Repository

Center for Machine Learning and Intelligent Systems

About Citation Policy Donate a Data Set Contact

Search

Repository Web

Google

View ALL Data Sets

Online News Popularity Data Set

Download: Data Folder, Data Set Description

Abstract: This dataset summarizes a heterogeneous set of features about articles published by Mashable in a period of two years. The goal is to predict the number of shares in social networks (popularity).

Data Set Characteristics:	Multivariate	**Number of Instances:**	39797	**Area:**	Business
Attribute Characteristics:	Integer, Real	**Number of Attributes:**	61	**Date Donated**	2015-05-31
Associated Tasks:	Classification, Regression	**Missing Values?**	N/A	**Number of Web Hits:**	361901

Source:

Kelwin Fernandes (kafc â€˜@â€™ inesctec.pt, kelwinfc â€™@â€™ gmail.com) - INESC TEC, Porto, Portugal/Universidade do Porto, Portugal.
Pedro Vinagre (pedro.vinagre.sousa â€™@â€™ gmail.com) - ALGORITMI Research Centre, Universidade do Minho, Portugal
Paulo Cortez - ALGORITMI Research Centre, Universidade do Minho, Portugal
Pedro Sernadela - Universidade de Aveiro

Data Set Information:

* The articles were published by Mashable (www.mashable.com) and their content as the rights to reproduce it belongs to them. Hence, this dataset does not share the original content but some statistics associated with it. The original content be publicly accessed and retrieved using the provided urls.
* Acquisition date: January 8, 2015
* The estimated relative performance values were estimated by the authors using a Random Forest classifier and a rolling windows as assessment method. See their article for more details on how the relative performance values were set.

Factor Information

61 columns in the dataset.

Non-predictive

1. url
2. timedelta

Category

1. data_channel_is_lifestyle
2. data_channel_is_entertainment
3. data_channel_is_bus
4. data_channel_is_socmed
5. data_channel_is_tech
6. data_channel_is_world

Self Reference

1. self_reference_min_shares
2. self_reference_max_shares
3. self_reference_avg_sharess

Content

1. n_tokens_title
2. n_tokens_content
3. n_unique_tokens
4. n_non_stop_words
5. n_non_stop_unique_tokens
6. num_hrefs
7. num_self_hrefs
8. num_imgs, num_videos
9. average_token_length
10. num_keywords

Keyword

1. kw_min_min
2. kw_max_min
3. kw_avg_min
4. kw_min_max
5. kw_max_max
6. kw_avg_max
7. kw_min_avg
8. kw_max_avg
9. kw_avg_avg

Weekday

1. weekday_is_monday
2. weekday_is_tuesday
3. weekday_is_wednesday
4. weekday_is_thursday
5. weekday_is_friday
6. weekday_is_saturday
7. weekday_is_sunday
8. is_weekend

Factor Information

61 columns in the dataset.

LDA
(Latent Dirichlet Allocation)
Topic

1. LDA_00
2. LDA_01
3. LDA_02
4. LDA_03
5. LDA_04

NLP

1. global_subjectivity
2. global_sentiment_polarity
3. global_rate_positive_words
4. global_rate_negative_words
5. rate_positive_words
6. rate_negative_words
7. avg_positive_polarity
8. min_positive_polarity
9. max_positive_polarity
10. avg_negative_polarity
11. min_negative_polarity
12. max_negative_polarity
13. title_subjectivity
14. title_sentiment_polarity
15. abs_title_subjectivity
16. abs_title_sentiment_polarity

Target Variable

1. shares

Define Problems

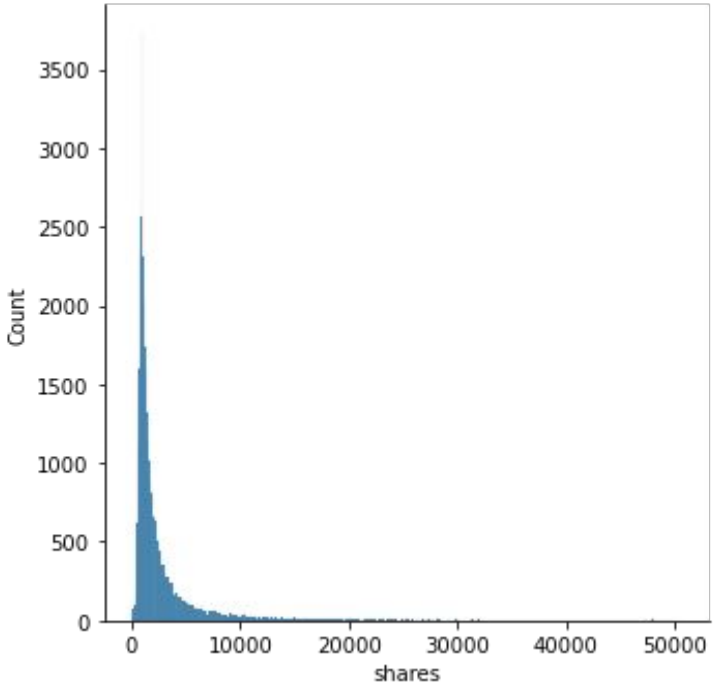
To predict the shares of online news.

Intuition of
Best Predictors And Performance

I guess the precision of models won’t be great, and it is hard to find a strong predictors.

ML Approach

1. Scale data with log transform
    a. The mean of shares is 2,869.
    b. The median of shares is 1,400.
    c. The 95th quantile is 10,800, and the 99th quantile is 31,663.



ML Approach

1. Scalr data with log transform
    a. The mean of shares is 2,869.
    b. The median of shares is 1,400.
    c. The 95th quantile is 10,800, and the 99th quantile is 31,663.
2. Build the model using linear regression, SVM, and XGBoost, and regard linear regression as a baseline.
3. Using GridSearchCV to find the best hyperparameters.

```
reg = LinearRegression().fit(X_train, y_train)
```

```
params = {'C': [1, 2],
          'degree': [3, 4],
          'epsilon':[0.1, 0.3]
         }
model = GridSearchCV(SVR(), params, cv=3)
model.fit(X_train, y_train)
```

```
params = {'eta': [0.05, 0.08],
          'min_child_weight': [1],
          'max_depth':[6, 8],
          'n_estimators':[100, 200],
         }
model = GridSearchCV(xgb.XGBRegressor(), params, cv=3)
model.fit(X_train, y_train)
```

Evaluate Performance - Linear Regression

```
reg = LinearRegression().fit(X_train, y_train)
reg.score(X_train, y_train)
```

```
0.11233745250530103
```

Evaluate Performance - SVM

```
params = {'C': [1, 2],
          'degree': [3, 4],
          'epsilon':[0.1, 0.3]
         }
model = GridSearchCV(SVR(), params, cv=3)
model.fit(X_train, y_train)
```

```
'params': [{'C': 1, 'degree': 3, 'epsilon': 0.1},
 {'C': 1, 'degree': 3, 'epsilon': 0.3},
 {'C': 1, 'degree': 4, 'epsilon': 0.1},
 {'C': 1, 'degree': 4, 'epsilon': 0.3},
 {'C': 2, 'degree': 3, 'epsilon': 0.1},
 {'C': 2, 'degree': 3, 'epsilon': 0.3},
 {'C': 2, 'degree': 4, 'epsilon': 0.1},
 {'C': 2, 'degree': 4, 'epsilon': 0.3}],
'split0_test_score': array([0.10536067, 0.11610535, 0.10536067, 0.11610535, 0.10817371,
       0.11875614, 0.10817371, 0.11875614]),
'split1_test_score': array([0.10428448, 0.11608638, 0.10428448, 0.11608638, 0.10745195,
       0.11874762, 0.10745195, 0.11874762]),
'split2_test_score': array([0.1124608 , 0.12368942, 0.1124608 , 0.12368942, 0.11600649,
       0.1275133 , 0.11600649, 0.1275133 ]),
'mean_test_score': array([0.10736865, 0.11862705, 0.10736865, 0.11862705, 0.11054405,
       0.12167235, 0.11054405, 0.12167235]),
'std_test_score': array([0.0036274 , 0.00357964, 0.0036274 , 0.00357964, 0.00387375,
       0.00413018, 0.00387375, 0.00413018]),
'rank_test_score': array([7, 3, 7, 3, 5, 1, 5, 1], dtype=int32)}
```

Evaluate Performance - XGBoost

```
params = {'eta': [0.05, 0.08],
          'min_child_weight': [1],
          'max_depth':[6, 8],
          'n_estimators':[100, 200],
         }
model = GridSearchCV(xgb.XGBRegressor(), params, cv=3)
model.fit(X_train, y_train)
```

```
'params': [{'eta': 0.05,
  'max_depth': 6,
  'min_child_weight': 1,
  'n_estimators': 100},
 {'eta': 0.05, 'max_depth': 6, 'min_child_weight': 1, 'n_estimators': 200},
 {'eta': 0.05, 'max_depth': 8, 'min_child_weight': 1, 'n_estimators': 100},
 {'eta': 0.05, 'max_depth': 8, 'min_child_weight': 1, 'n_estimators': 200},
 {'eta': 0.08, 'max_depth': 6, 'min_child_weight': 1, 'n_estimators': 100},
 {'eta': 0.08, 'max_depth': 6, 'min_child_weight': 1, 'n_estimators': 200},
 {'eta': 0.08, 'max_depth': 8, 'min_child_weight': 1, 'n_estimators': 100},
 {'eta': 0.08, 'max_depth': 8, 'min_child_weight': 1, 'n_estimators': 200}],
'split0_test_score': array([0.16577636, 0.17132127, 0.1644699 , 0.16718769, 0.16800084,
       0.16684987, 0.16490341, 0.15674626]),
'split1_test_score': array([0.15072261, 0.15640715, 0.14901039, 0.15485203, 0.15425099,
       0.15511687, 0.15342615, 0.15147335]),
'split2_test_score': array([0.15449637, 0.16000671, 0.15477105, 0.15687225, 0.15737878,
       0.15442976, 0.15378223, 0.14524207]),
'mean_test_score': array([0.15699845, 0.16257837, 0.15608378, 0.15963732, 0.15987687,
       0.15879883, 0.1573706 , 0.15115389]),
'std_test_score': array([0.00639527, 0.00635441, 0.00637921, 0.00540225, 0.00588472,
       0.00569985, 0.00532849, 0.004702  ]),
'rank_test_score': array([6, 1, 7, 3, 2, 4, 5, 8], dtype=int32)}
```

Mean Squared Error

Linear Regression

```
mean_squared_error(y_test_inverse, y_pred_inverse)
```

```
13801812.921976253
```

SVM

```
mean_squared_error(y_test_inverse, y_pred_inverse)
```

```
14106431.710427659
```

XGBoost

```
mean_squared_error(y_test_inverse, y_pred_inverse)
```

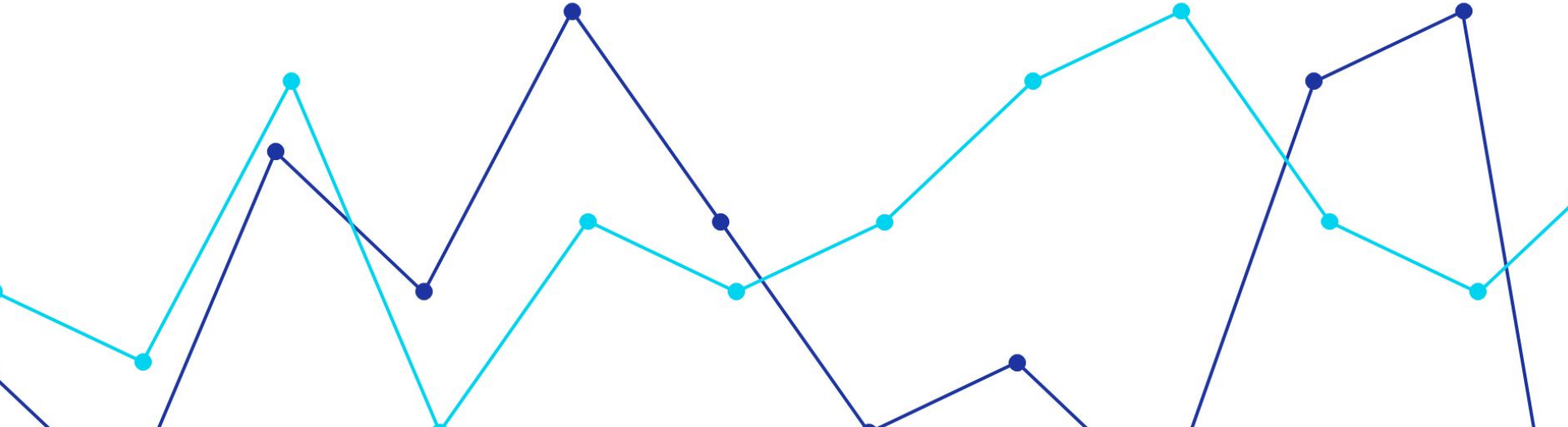
```
13502546.775470285
```

Feature Importance

Features are sorted descendingly by importance.

Feature	Importance
data_channel_is_tech	0.086420
data_channel_is_world	0.079242
kw_max_avg	0.077997
self_reference_min_shares	0.048806
data_channel_is_entertainment	0.048475
data_channel_is_socmed	0.044412
weekday_is_saturday	0.043352
weekday_is_sunday	0.039722
self_reference_max_shares	0.036566
kw_min_avg	0.031878
data_channel_is_bus	0.023367
num_videos	0.020041
num_hrefs	0.019583
n_unique_tokens	0.017763
num_imgs	0.017616
min_positive_polarity	0.017499
kw_avg_max	0.017123

Conclusion

1. This is a complicated real world problem, so it is hard to predict. The potential solutions are to collect more data or apply a more complicated model.
2. In this case, the performance of XGBoost is better than SVM and linear regression.

Thank You